LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 – ENKAPSULASI



NAMA: ANDI MUHAMMAD ABID JAYA

NIM: 103132400014

KELAS: S1-SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Teori

Dalam modul ini mempelajari beberapa dasar teori dasar sebagai berikut:

1. Java Modifier, Modifier digunakan untuk mengatur hak akses dan sifat pada kelas, antarmuka, method, maupun variabel. Dalam pemrograman berbasis objek, hal ini sangat berguna untuk enkapsulasi (menyembunyikan data) dengan cara membuat atribut bersifat private dan menyediakan method public (seperti *getter/setter*) untuk mengaksesnya.
2. Constructor, Constructor adalah method khusus yang bertugas hanya untuk menciptakan atau menginstansiasi objek. Aturan utamanya: namanya harus persis sama dengan nama kelas, tidak boleh memiliki nilai kembalian (bahkan tanpa void), dan bisa dibuat lebih dari satu (*overloading*) asalkan parameternya berbeda-beda.
3. Package, Package adalah cara untuk mengelompokkan kelas-kelas ke dalam direktori agar lebih rapi dan menghindari bentrok atau konflik penamaan method. Aturannya sederhana: jika kita ingin memakai kelas yang berada di *package* yang berbeda, kita wajib menggunakan perintah

B. Guided

1. Guided 1

```
// nama package: Manusia    nama file: Manusia.java

package com.abid.guided.gided1.Manusia;

public class Manusia {
    // definisi atribut
    private String nama;
    private int umur;

    // constructor

    public Manusia() {
    };

    public Manusia(String nama) {
        this.nama = nama;
    }
```

```
    public Manusia(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    // definisi Method
    // metod setter
    public void setNama(String a) {
        nama = a;
    }

    public void setUmur(int a) {
        umur = a;
    }

    // metod getter
    public String getNama() {
        return nama;
    }

    public int getUmur() {
        return umur;
    }
}
```

```
// nama package: main    nama file: demoManusia.java

        package com.abid.guided.gided1.main;

        import com.abid.guided.gided1.Manusia.Manusia;

        public class demoManusia {
            public static void main(String[] args) {
                Manusia arrMns[] = new Manusia[3];

                // constructor
                Manusia objMns1 = new Manusia();
                Manusia objMns2 = new Manusia("joy");
                Manusia objMns3 = new Manusia("jooo", 88);

                arrMns[0] = objMns1;
                arrMns[1] = objMns2;
```

```
            arrMns[2] = objMns3;

            for (int i = 0; i < 3; i++) {
                System.out.println("nama: " + arrMns[i].getNama());
                System.out.println("umur: " + arrMns[i].getUmur());
                System.out.println();
            }
        }
    }
```

Output :

```
PS D:\SEMESTER 5\PEMROGRAMAN BERBASIS OBJEK\TEMPLATE REPO\praktikum-pbo-ps1ds-05-reg-01>  & 'C:\
4\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\S
kum-pbo-ps1ds-05-reg-01\modul-04-enkapsulasi\project\modul4\target\classes' 'com.abid.guided.gid
nama: null
umur: 0

nama: joy
umur: 0

nama: jooo
umur: 88
```

2. Guided 2

```
// nama package: hargapulsa    nama file: hargapulsa.java

package com.abid.guided.gided2.hargapulsa;

public class hargapulsa {
    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}
```

```
// nama package: hargatoken    nama file: hargatoken.java

        package com.abid.guided.gided2.hargatoken;

        public class hargatoken {
            public void info() {
                System.out.println("Harga Token");
                System.out.println("Rp. 20000");
                System.out.println("Rp. 50000");
                System.out.println("Rp. 100000");
                System.out.println("Rp. 200000");
                System.out.println("Rp. 500000");
            }
        }
```

```
// nama package: main    nama file: main.java

        package com.abid.guided.gided2.main;

        import com.abid.guided.gided2.hargapulsa.hargapulsa;
        import com.abid.guided.gided2.hargatoken.hargatoken;

        public class main {
            public static void main(String[] args) {

                hargatoken objectToken = new hargatoken();
                objectToken.info();

                hargapulsa objectPulsa = new hargapulsa();
                objectPulsa.info();
            }
        }
```

Output :

```
4\target\classes' 'com.abid.guided.gided2.main.main'
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
PS D:\SEMESTER 5\PEMROGRAMAN BERBASIS OBJEK\TEMPLATE REPO\praktikum-pbo-ps1ds-05-reg-01>
```

C. Unguided

1. Soal Unguided

Soal Unguided — Modul 4: Enkapsulasi

Sistem Pencatatan Dataset

Kamu sedang membangun program kecil untuk mencatat dan memeriksa kualitas dataset sebelum dianalisis, seperti tahap awal pengecekan *missing value*. Setiap dataset punya nama, jumlah baris, jumlah kolom, dan jumlah sel yang kosong (*missing*).

Petunjuk Umum

Buat **3 package** dengan struktur berikut:

```
Source Packages
├── model
│   └── Dataset.java
├── laporan
│   └── LaporanDataset.java
└── main
    └── Main.java
```

1. Class `Dataset` (package `model`)

Atribut (implementasikan enkapsulasi):

Atribut	Tipe	Keterangan
`nama`	`String`	Nama dataset
`jumlahBaris`	`int`	Banyak baris (observasi)
`jumlahKolom`	`int`	Banyak kolom (fitur)
`jumlahMissing`	`int`	Banyak sel yang kosong

Ketentuan tambahan:

- Buat **konstanta** `BATAS_MISSING` bernilai `5.0` (persen). Dataset dengan missing diatas batas ini dianggap perlu dibersihkan.
- Buat **variabel milik class** yang menghitung total objek `Dataset` yang telah dibuat, beserta **method** untuk ambil nilainya tanpa perlu membuat objek.

Constructor (ada 3, overloading):

1. Tanpa parameter
2. Hanya parameter `nama`
3. Lengkap: `nama`, `jumlahBaris`, `jumlahKolom`, `jumlahMissing`

Setiap constructor harus menambah hitungan total dataset.

Method:

- Getter dan setter untuk semua atribut.
- Pada setter `jumlahBaris`, `jumlahKolom`, dan `jumlahMissing`, buat agar nilai tidak berubah.
- `getPersentaseMissing()` mengembalikan `double`: persentase sel kosong dibanding total sel (`baris × kolom`). Jika total sel = 0, kembalikan `0`.
- `perluDibersihkan()` mengembalikan `boolean`: `true` jika persentase missing melebihi `BATAS_MISSING`.

2. Class `LaporanDataset` (package `laporan`)

Buat method `cetak` yang menerima satu objek `Dataset` dan menampilkan laporannya dengan format:

```
=== Laporan Dataset ===
Nama         : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing      : 866 sel (8.10%)
Status       : Perlu dibersihkan
```

Status berisi `Perlu dibersihkan` atau `Bersih` sesuai hasil dari `perluDibersihkan()`.

3. Class `Main` (package `main`)

Buat tiga objek, masing-masing dengan constructor berbeda:

Objek	Cara pembuatan	Data
1	Constructor tanpa parameter, lalu isi memakai setter	Titanic, 891 baris, 12 kolom, 866 missing
2	Constructor 1 parameter	Wine Quality
3	Constructor lengkap	Iris, 150 baris, 5 kolom, 0 missing

Simpan ketiganya dalam **array**, cetak laporannya dengan perulangan, lalu tampilkan di akhir:

```
Total dataset dibuat : 3
```

Jawaban :

```
// nama package: model    nama file: Dataset.java

        package com.abid.unguided.model;

        public class Dataset {
        // Atribut (enkapsulasi dengan access modifier private)
            private String nama;
            private int jumlahBaris;
            private int jumlahKolom;
            private int jumlahMissing;

            // Konstanta batas missing value (5.0%)
            public static final double BATAS_MISSING = 5.0;

            // Variabel milik class untuk menghitung total objek yang dibuat
```

```
    private static int totalDataset = 0;

    // --- Constructor Overloading ---

    // 1. Constructor tanpa parameter
    public Dataset() {
        totalDataset++; // Menambah hitungan
    }

    // 2. Constructor dengan 1 parameter
    public Dataset(String nama) {
        this.nama = nama;
        totalDataset++; // Menambah hitungan
    }

    // 3. Constructor lengkap
    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int jumlahMissing) {
        this.nama = nama;
        // Memastikan nilai awal tidak diset negatif
        if (jumlahBaris >= 0) this.jumlahBaris = jumlahBaris;
        if (jumlahKolom >= 0) this.jumlahKolom = jumlahKolom;
        if (jumlahMissing >= 0) this.jumlahMissing = jumlahMissing;
        totalDataset++;
    }

    // --- Method ---

    public static int getTotalDataset() {
        return totalDataset;
    }

    // Getter untuk semua atribut
    public String getNama() { return nama; }
    public int getJumlahBaris() { return jumlahBaris; }
    public int getJumlahKolom() { return jumlahKolom; }
    public int getJumlahMissing() { return jumlahMissing; }

    // Setter untuk atribut nama
    public void setNama(String nama) {
        this.nama = nama;
    }

    // Setter dengan validasi agar nilai tidak negatif
    public void setJumlahBaris(int jumlahBaris) {
        if (jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
```

```
        }
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (jumlahKolom >= 0) {
            this.jumlahKolom = jumlahKolom;
        }
    }

    public void setJumlahMissing(int jumlahMissing) {
        if (jumlahMissing >= 0) {
            this.jumlahMissing = jumlahMissing;
        }
    }

    // Mengembalikan persentase sel kosong
    public double getPersentaseMissing() {
        int totalSel = jumlahBaris * jumlahKolom;
        if (totalSel == 0) {
            return 0.0; // Mencegah error pembagian dengan nol
        }
        return ((double) jumlahMissing / totalSel) * 100.0;
    }

    // Mengecek apakah dataset perlu dibersihkan
    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}
```

```
// nama package: laporan    nama file: LaporanDataset.java

package com.abid.unguided.laporan;

import com.abid.unguided.model.Dataset;

public class LaporanDataset {
    // Method untuk menampilkan laporan
    public void cetak(Dataset d) {
        System.out.println("=== Laporan Dataset ===");
        // Menangani kasus jika nama belum diisi (null)
        System.out.println("Nama         : " + (d.getNama() == null ? "-" :
d.getNama()));
```

```
        System.out.println("Jumlah Baris : " + d.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + d.getJumlahKolom());

        // Menggunakan printf untuk format angka di belakang koma
        System.out.printf("Missing      : %d sel (%.2f%%)\n", d.getJumlahMissing(),
    d.getPersentaseMissing());

        String status = d.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih";
        System.out.println("Status       : " + status);
        System.out.println(); // Baris kosong sebagai pemisah antar laporan
    }
}
```

```
// nama package: main    nama file: Main.java

    package com.abid.unguided.main;

    import com.abid.unguided.laporan.LaporanDataset;
    import com.abid.unguided.model.Dataset;

    public class Main {
        public static void main(String[] args) {
            // Objek 1: Constructor tanpa parameter, isi memakai setter
            Dataset data1 = new Dataset();
            data1.setNama("Titanic");
            data1.setJumlahBaris(891);
            data1.setJumlahKolom(12);
            data1.setJumlahMissing(866);

            // Objek 2: Constructor 1 parameter
            Dataset data2 = new Dataset("Wine Quality");
            // Karena baris dan kolom tidak diset, nilainya default 0

            // Objek 3: Constructor lengkap
            Dataset data3 = new Dataset("Iris", 150, 5, 0);

            // Menyimpan ketiga objek ke dalam array
            Dataset[] daftarDataset = {data1, data2, data3};
```

```
        // Instansiasi objek pelapor
        LaporanDataset pelapor = new LaporanDataset();

        // Mencetak laporan dengan perulangan
        for (int i = 0; i < daftarDataset.length; i++) {
            pelapor.cetak(daftarDataset[i]);
        }

        // Menampilkan total dataset dengan memanggil method static
        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}
```

Output :

```
=== Laporan Dataset ===
Nama         : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing      : 866 sel (8.10%)
Status       : Perlu dibersihkan

=== Laporan Dataset ===
Nama         : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing      : 0 sel (0.00%)
Status       : Bersih

=== Laporan Dataset ===
Nama         : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing      : 0 sel (0.00%)
Status       : Bersih

Total dataset dibuat : 3
```

Penjelasan :

File Dataset.java berisi kelas Dataset dengan instance field private (nama, jumlahBaris, jumlahKolom, jumlahMissing) untuk enkapsulasi, serta class field static final BATAS_MISSING dan static totalDataset. Objek dibuat melalui tiga constructor (overloading) yang otomatis menambah hitungan totalDataset. Kelas ini dilengkapi getter, setter, serta method getPersentaseMissing() dan perluDibersihkan() untuk evaluasi data.

File LaporanDataset.java memiliki method cetak() yang khusus bertugas memformat output laporan dari objek Dataset. Sementara itu, file Main.java menginisialisasi tiga objek menggunakan jenis constructor yang berbeda dan menyimpannya ke dalam array, lalu memanggil method cetak() menggunakan perulangan untuk menampilkan semuanya.

Di akhir, Main.java memanggil method static getTotalDataset(). Nilai yang tercetak adalah 3 karena variabel totalDataset bersifat static. Hal ini berarti variabel tersebut dimiliki bersama oleh kelas, sehingga nilainya akan selalu terakumulasi setiap kali constructor dipanggil untuk membuat objek baru.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan konsep Enkapsulasi (Data Hiding) sangat krusial dalam melindungi atribut kelas dari modifikasi yang tidak sah atau tidak valid dari luar. Dengan menetapkan access modifier private pada atribut dan menyediakan antarmuka melalui method public seperti getter dan setter, integritas data dapat dijaga, misalnya dengan menambahkan validasi untuk mencegah input nilai negatif pada baris dan kolom dataset. Selain itu, penggunaan class field dengan modifier static terbukti sangat berguna untuk membuat variabel yang dibagikan ke semua objek, seperti penghitung total dataset, sementara modifier static final sangat efektif untuk membuat konstanta permanen.

Praktikum ini juga membuktikan kelebihan dari penerapan Constructor Overloading. Pembuatan lebih dari satu constructor dalam sebuah kelas memberikan fleksibilitas tinggi, sehingga objek dapat diinisialisasi dengan berbagai cara tergantung pada kelengkapan parameter data yang tersedia saat objek tersebut diciptakan. Di samping itu, pembagian kode ke dalam beberapa package yang berbeda (seperti model, laporan, dan main) menunjukkan bagaimana manajemen direktori mampu membuat struktur program menjadi jauh lebih rapi dan modular. Hal ini juga melatih pemahaman bahwa penggunaan kelas yang berada di luar package asalnya mewajibkan pemanggilan menggunakan perintah import agar program dapat berjalan dengan baik.